

ECE 385
Spring 2025
Final Lab Report

Jetpack Joyride (385 Edition)

Rohan Shah, Anish Nagar
rohan20, anishn4
Friday, May 9th, 2025, 11:00 pm
Dongming Liu, dl35@illinois.edu

1. Introduction

For our final project, we created a game, very similar to 'Jetpack Joyride,' which controls a character navigating through an obstacle course of zappers, missiles, and other modules. Building off of Lab 6, we were able to use VGA and display the game on the screen, and use the keyboard to control our character, dodging each of the obstacles we put in our version of this game. We utilized a state machine, sprite ROMs, and various other modules to incorporate a fully-functional concept of this game, which was playable and without many bugs. Moreover, we were able to implement collision detection to add several states to the game, including a loading screen, an in-game state, and a dead state, which would allow the player to restart the whole game or jump back into another game based on keyboard input.

2. Written Description of Final System

BuildUp from Lab 6.2

In Lab 6.2, we were able to show a ball moving on the screen, controlled by the AXI GPIO blocks we added in our block design, interfacing with the keyboard. We had a color palette module that would render specific colors on the screen based on an 'on' signal, and move that ball with the keyboard. In our final project, we implemented a similar aspect, but much more complicated, as now we have several states controlled by a keyboard, along with the implementation of gravity. Our new microcontroller now utilizes 128kB of memory compared to the previous 32kB, and we controlled various sprites with a similar build to that of Lab 6.2. However, the overall block design stayed the same, aside from the hardware descriptions we wrote as part of this final project.

Implementation of BRAM

We created a BRAM block in our top-level diagram to store our character and background data. Every time we uploaded a new sprite image, obstacle, or background, we stored it in BRAM. Each of the sprite ROMs was stored individually in BRAM, which we ultimately ran out of. To optimize our memory, we had to downsize many of our images to a lower resolution. This reduced the BRAM usage of each ROM to 1/16th of its original size, allowing us to implement additional sprites and obstacles on the screen.

Sprite ROM and Sprite Imaging

With a provided image to COE converter, the process started with initially downloading an image from online, and utilizing the Python script to create our .sv file, color palette, and the coefficient (COE) file. Once this process was done with the tool, we had to instantiate a ROM for this specific sprite, which would be an instance of the Block RAM (BRAM). To properly create the BRAM module for our ROM, we had to set a specific width and depth for our COE file, which was provided when generating the sprite modules. Furthermore, we had to upload the COE file within the ROM under 'Load Init File.' Finally, renaming the sprite allowed us to utilize the ROM mapped to a specific sprite.

As we now have a specific color palette mapped to a ROM and a sprite, we can display the sprite on the screen in a desired position by reading from the ROM.

We deduced an equation to read from our sprite ROM, allowing us to fetch the corresponding ROM address for our sprite. The line below is an example of converting our pixels to a ROM address to correctly display the colors on the screen.

```
assign rom_address = spritex + (45 * spritey);
```

However, to determine whether we want the sprite to show up or not based on pixels, we utilized the formula:

```
assign on = (spritex < 45) && (spritey < 57) && (DrawX >= BallX) && (DrawY >= BallY).
```

This formula was specifically utilized for our 'Barry' sprite, or the character that we moved through the obstacles. By checking the dimensions of the sprite, we know whether or not to display the colors of the sprite in the specific area. If we do map the colors, then we set it to the RGB palette of Barry, else the background. The background always remained on, and the sprites were overriding the RGB values for those specific pixels. For our zapper and missile, we utilized the same idea, but changing the dimensions to the respective ones for each sprite.



Figure 1 - Loading screen

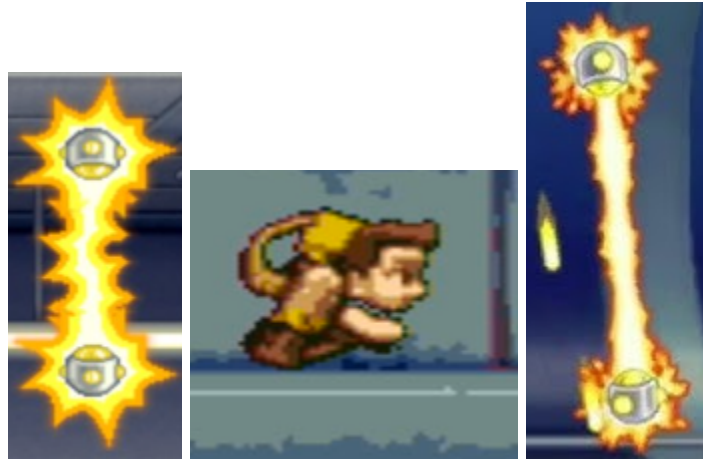


Figure 2 - The Zapper sprites and the Barry sprite

Sprite Movement

To make our sprites move across the screen, we implemented a control unit for each one, along with one central 'movement timer' module that sent out a pulse for each movement to occur. This movement counter had a 24-bit counter that was essentially powered by the inputted clock, in this case 25 MHz. If the counter modded with another number, which we set as 800000, was ever 0, then it would send out a singular 'move enable' pulse. This pulse allowed our objects to move to the left each time. If we wanted to speed up our objects, we could reduce the 800000 counter checks we added, and increase the number to slow them down. This changes the rate of the move enable pulse.

For the control unit itself, we set an initial speed for each object, along with an initial x and initial y position, all of which are hardcoded to set values. However, we included a computation for the y position after reset, which varied for each of the zappers and the missile itself. The missile's y position was updated every few seconds with our character's (Barry's) y position, which allowed the missile to 'target' our character, increasing game difficulty. The y position of the zappers was pseudo-random, meaning we had a set of specified positions, calculated in such a way that they appear randomly across the screen. This allows the game to have a 'random' effect and make it as close to the real version as possible.

To properly utilize our pseudo-randomness, we utilized the counter from our movement timer, and took different bits of this counter as the y-value, while also setting a specific boundary for the top and bottom of the y-values. This created the random effect, setting the y position of zappers, as they moved to the left (x-position) at a steady rate.

FSM (Finite State Machine) Implementation

The control unit we used is a Moore finite state machine which switches between three main states of Loading, Game, and Dead. When reset, we first show the Loading state, which asserts the loading_screen output to show the loading screen image. Once the player presses the start

key, the FSM goes into the Game state, and then the actual game, showing the sprite and obstacle,s is shown.

While in the Game state, the finite state machine checks for collision detection between the sprite image and the obstacles. If there is an overlap between the sprite and an obstacle, then the FSM goes into the Dead state because the player has died. In this state, we show the dead screen since the game is over. Then once again, if the key 'R' is pressed, the game goes back into the Game state; otherwise, if 'W' is pressed, the game goes back into the Loading state. That completes the entire FSM of the game.

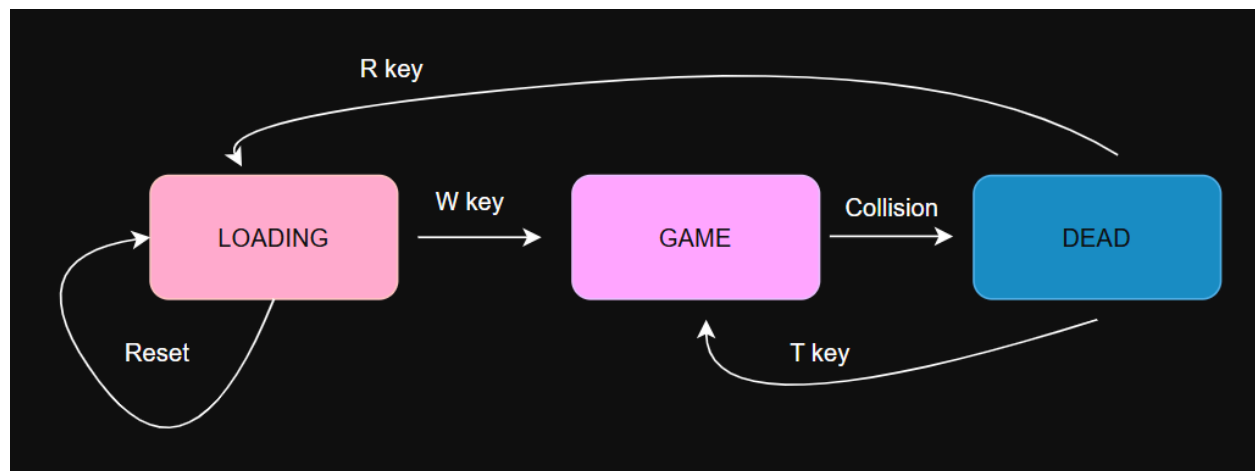


Figure 3 - Finite State Machine

Implementation of Gravity and Keyboard Interfacing

The way we made the physics for gravity is by continuously pulling down the sprite if nothing is pressed. Whenever the user presses the spacebar, we subtract the value 4 from the y coordinate of the character. This allowed us to show the jetpack moving upward. Otherwise, the value of 1 was always added to the Y value of the sprite. We also clamped the velocity between 6 and negative 6 so that the fall and rise speeds wouldn't get too high or low. We also set up bounds at the top and the bottom, so that the sprite character can't leave the boundaries of the screen.

The way we detect values from the keyboard is through the MicroBlaze. Similar to how we did in lab 6.2, we used a USB host interface that can decode scan codes from an attached keyboard. The codes are written into 32-bit AXI GPIO registers, which take the lower 8 bits of the keycode, which we use to check for values such as the spacebar, 'W', and 'R'.

How Collision Detection Works

```
GAME: begin
  if (( //barry height and width are both 40 - made a bit smaller for a better hit box
    (BallX + 10'd40 > ZapperX) && (BallX < ZapperX + 10'd31) && //zapper width 1
    (BallY + 10'd40 > ZapperY) && (BallY < ZapperY + 10'd96)) //zapper height 1
  || ((BallX + 10'd40 > zap2x) && (BallX < zap2x + 10'd55) && //zapper width 2
    (BallY + 10'd40 > zap2y) && (BallY < zap2y + 10'd172)) || //zapper height 2
    ((BallX + 10'd40 > misx) && (BallX < misx + 5'd20) && //missile width
    (BallY + 10'd40 > misy) && (BallY < misy + 5'd20)) //missile height
  ) //bounds and collision detection for all the modules, the missiles, zapper etc....

  begin
    next_state = DEAD;
  end
end
```

Figure 4 - Collision detection code

To implement the collision detection, we checked to ensure an overlap between our character and any obstacle that it encountered in its path. We tweaked the bounds for each of the objects for a much closer collision detection, rather than colliding with the rectangle outside the bounds of each sprite, which included a black background as well. If there was an overlap in the x and y coordinates of a corresponding item, then we proceed to the 'DEAD' state, which signals the end of the game. This was done for both zappers, along with the missile, ensuring proper functionality for this game.

3. Block Diagram

- a. This diagram should represent the placement of all your modules in the top level.

4. Module Descriptions

Module: mb_usb_hdmi_top

Inputs: Clk, reset_rtl_0, gpio_usb_int_tri_i, usb_spi_miso, uart_rtl_0_rxd,

Outputs: gpio_usb_rst_tri_o, usb_spi_mosi, usb_spi_sclk, usb_spi_ss, uart_rtl_0_txd, hdmi_tmds_clk_n, hdmi_tmds_clk_p, [2:0]hdmi_tmds_data_n, [2:0]hdmi_tmds_data_p, [7:0] hex_segA, [3:0] hex_gridA, [7:0] hex_segB, [3:0] hex_gridB

Description: The module is the top level of the entire project. It instantiates all of the other modules we have and connects everything on a hardware level. We have a MicroBlaze processor handling the USB keyboard with a UART. We have a VGA controller using the drawX and drawY signals and VGA display signals. Another big part of this module is instantiating the control unit, which has all the transitions between states.

Purpose: The purpose of this module is to put everything together. It makes the clock signals which are then used to control everything else in the game. It is also responsible for telling the MicroBlaze to use HDMI and USB for display graphics and keyboard input.

Module: barry_example

Inputs: vga_clk, keycode, firer, fireg, fireb, DrawX, DrawY, BallX, BallY, bg_red, bg_green, bg_blue, mr, mg, mb, load_r, load_g, load_b, in_game, loading_screen, ZapperX, ZapperY, zapper_r, zapper_g, zapper_b, dead_r, dead_g, dead_b, dead

Outputs: red, green, blue, spritex, spritey

Description: This module is one of the top levels for our project. It returns the sprite and spritey variables, which are responsible for the placement of Barry on the screen. These variables also help come up with the logic for all the other obstacles. We use the x and y coordinates to come up with a ROM address, and we fetch the sprite graphics from rgb signals from the barry_palette module. We finally output the RGB values into the final four-bit outputs.

Purpose: The purpose of the module is to synchronize the pixel clock for all the components in our project. This module handles sprite addressing, ROM addressing, and RGB values for the separate components, depending on what state we are in.

Module: bg_palette

Inputs: [2:0] index

Outputs: [3:0] red, green, blue

Description: This module is to look up colors for the Barry sprite. It was made from the Image to the COE Python library. It uses a 3-bit input index to select and output RGB values.

Purpose: The purpose is to show the correct colors for the Barry sprite. The values in the palette represent the different colors in the image.

Module: `bg_example`

Inputs: `vga_clk`, `DrawX`, `DrawY`, `blank`

Outputs: `red`, `green`, `blue`

Description: This module is for rendering the background. It does so by taking the pixel coordinates `DrawX` and `DrawY` as inputs, and computes a ROM address, which will get the RGB values from the pixel ROM. Then the background palette is responsible for making the four-bit RGB values, which show the colors of the background. Then the palette colors are outputted.

Purpose: The purpose of this module is to centralize all the logic needed to display our background image. This module does address calculation, palette mapping, and control for when to show up.

Module: `bg_palette`

Inputs: `[2:0]` `index`

Outputs: `[3:0]` `red`, `green`, `blue`

Description: This module stores the array that has the colors for the background. It takes in a 3-bit input and outputs 4-bit RGB values.

Purpose: The purpose of the module is to contain all the data for the colors for our main background while in the game state.

Module: `dead_example`

Inputs: `vga_clk`, `[9:0]` `DrawX`, `DrawY`, `blank`, `dead`

Outputs: `[3:0]` `red`, `green`, `blue`

Description: This module renders the background for the death screen. This happens when the `dead` signal is asserted in the control module. The module also uses the inverted VGA clock edge to read a 3-bit pixel index from the `dead_rom` signal. This module uses the RGB values from the `dead_palette`.

Purpose: Again, the purpose is to display a dead game background once the character dies in the game.

Module: dead_palette

Inputs: [2:0] index

Outputs: [3:0] red, green, blue

Description: This module is similar to the other palette modules since it holds an array with all the colors the background should show once the character is dead. It takes a 3-bit index and outputs RGB values to be shown on the screen.

Purpose: The purpose is to convert the 3-bit pixel indices coming from the dead_rom into RGB values, which show a dead screen.

Module: vga_controller

Inputs: pixel_clk, reset

Outputs: hs, vs, active_nblank, sync, [9:0] drawX, drawY

Description: This module generates all of the timing signals and pixel coordinates to show our 640 by 480 VGA display. On each rising edge of the pixel_clk, we increment the horizontal and vertical counters. Signals drawX and drawY are the positions on the screen of where we are. Signals hs and vs are the horizontal and vertical sync signals.

Purpose: The purpose of the module is to display VGA graphics to the screen. It is also used to sync all the signals that are being used to display this output.

Module: ball

Inputs: Reset, frame_clk, [7:0] keycode

Outputs: [9:0] BallX, [9:0] BallY, BallS

Description: This module is taken from lab 6.2. The purpose is to do the vertical motion of the game, and also the gravity and physics for the sprite. All the signals get updated on each frame_clk. The sprite maintains a vertical velocity and x-axis position. When the spacebar is pressed, we subtract a value from our vertical position, making the character go upwards. If nothing is pressed, the character drops downward.

Purpose: The purpose is to control the Barry sprite. This includes physics for the jetpack to shoot the sprite upwards, and also the gravity on the player when nothing is pressed.

Module: zapper_example

Inputs: vga_clk, DrawX, DrawY, ZapperX, ZapperY, in_game,

Outputs: zapper_r, zapper_g, zapper_b, zapper_on

Description: This module is dedicated to creating the zapper object, which is used as an obstacle if the character collides with the zapper. In this module, we synchronize the position of ZapperX and ZapperY. We compute the coordinates of the zapper, and we generate an on flag whenever the zapper object is active. We also use the zapper_palette to get the colors for the image.

Purpose: The purpose of this module is to create a zapper object and move it across a screen. The object is used to make the game harder for the Barry sprite to dodge.

Module: zapper_palette

Inputs: [2:0] index

Outputs: [3:0] red, green, blue

Description: The palette serves the same purpose as the other palette modules. The point of this module is to create an array of colors that can be used as RGB values to make the zapper object. It takes a 3-bit input and outputs corresponding RGB values.

Purpose: The purpose is to get 4-bit RGB signals that can be displayed, showing a zapper object.

Module: loadingscreen_example

Inputs: vga_clk, [9:0] DrawX, DrawY, blank, loading_screen

Outputs: [3:0] red, green, blue

Description: This is a VGA-synchronized renderer that shows a loading screen background before playing the game. We downsampled the image so that we could fit it in BRAM. The way it works is we compute a ROM address and use the inverted VGA signal to fetch a pixel index. This index gets mapped through a palette, which has all the information for the background.

Purpose: The purpose of this module is to provide a background that shows a loading screen while the player waits before playing the game.

Module: loadingscreen_palette

Inputs: [2:0] index

Outputs: [3:0] red, green, blue

Description: This module contains an array with all the color data for the loading screen background. It is stored in an array, and the module takes in a 3-bit index and outputs a 4-bit RGB value.

Purpose: The purpose is to store all the colors for the loading screen. That way, we can make our project more like the original Jetpack Joyride game.

Module: control_unit

Inputs: reset, clk, [7:0] keycode, [9:0] ZapperX, ZapperY, [9:0] zap2x, zap2y, [9:0] spritex, spritey, [9:0] BallX, BallY, misx, misy,

Outputs: player_dead, loading_screen, in_game

Description: The control unit is responsible for transitioning from different states. This is a finite state machine with three states, which are loading, game, and dead. We set up logic for when to go to which state, and it depends on when certain keys are pressed. When we are in the game state, once the player dies and the player_dead signal is asserted, we go to the dead state. We also set up logic for if Barry hits the zappers here to change states.

Purpose: This module is crucial because it is an FSM that controls the entire process of the game.

Module: movement_timer

Inputs: clk, reset

Outputs: move_enable, [23:0] counter_out

Description: The movement_timer module is what helps us generate pseudo-random numbers for the zapper, along with increasing the zapper's speed over time. The counter is initialized to a value of 5, and it increases as time goes on.

Purpose: The output of this module is helpful in all other modules for determining random values or changing the speeds of objects.

Module: zapper_controller

Inputs: clk, reset, move_enable, [23:0] counter,

Outputs: [9:0] ZapperX, [9:0] ZapperY, in_game

Description: This module sets the motion for the zapper. The horizontal speed has a set value and moves from right to left, but we also use values from the counter; that way, as time goes on, the zapper can increase its speed, making the game harder. When the Zapper reaches the very left edge, it spawns again on the right side at a pseudo-random time and location on the y-axis.

Purpose: The purpose of this module is to move the zapper obstacle that the character has to dodge.

C Code Implementation

The C code we utilized as part of our final project was the same as from Lab 6.2, with the same functions filled out:

```
void MAXreg_wr(BYTE reg, BYTE val)
BYTE* MAXbytes_wr(BYTE reg, BYTE nbytes, BYTE* data)
BYTE MAXreg_rd(BYTE reg)
BYTE* MAXbytes_rd(BYTE reg, BYTE nbytes, BYTE* data)
```

Each of these functions is crucial for a correct interaction between the keyboard and our design, as we need to read and write each keycode using the SPI protocol. As a high-level description of each function:

void MAXreg_wr(BYTE reg, BYTE val)

This function performs a single host register write, which means to write the val byte to the provided reg on the MAX3421E chip. We then utilize the send and receive buffers to read and store the necessary values, eventually deinitializing the chip after a complete process. We control the Universal Serial Bus (USB) host through this process.

BYTE* MAXbytes_wr(BYTE reg, BYTE nbytes, BYTE* data)

This function performs a multiple-byte write and we return a memory pointer after the last operation. This is similar to the single byte process above, but the sizes of the buffer changes, along with being able to process all data bytes. We check the value of the status and ultimately deselect the chip as we previously did. We can send larger chunks of data through this transfer protocol.

BYTE MAXreg_rd(BYTE reg)

This function does a single host read, which utilizes the buffer, similar to the single host write. However the layout of the buffer is different in this, and now we are able to read values by having dummy values within the buffer. This uses the XSpi transfer function, and is important as we need to get the current value of a specified register for this protocol for our keyboard and peripheral.

BYTE* MAXbytes_rd(BYTE reg, BYTE nbytes, BYTE* data)

This function does a multiple-byte read, similar to the processes we did above. We need to store several items that we read, but this is repeated several times to set the value to our received value. Due to this function, we can read several bytes through one instantiation of the XSpi Transfer function we were provided.

5. Testing

In order to test our lab, we did not need to utilize any waveforms or simulations. Instead, our testing was done through viewing our output on the screen, especially since we had Lab 6.2 as a baseline. This made it much easier to visualize the issue and then reference the lab to see if we performed our color mapping or VGA interactions incorrectly. A specific issue we ran into was reading from the sprite ROMs incorrectly, not implementing a long enough delay to match the delay of the ROM. We added a delay of only one clock cycle, but this did not work well, producing random lines next to our sprites, signaling incorrect ROM readings. To combat this, we added an additional delay state in our modules, and this fixed our issue.

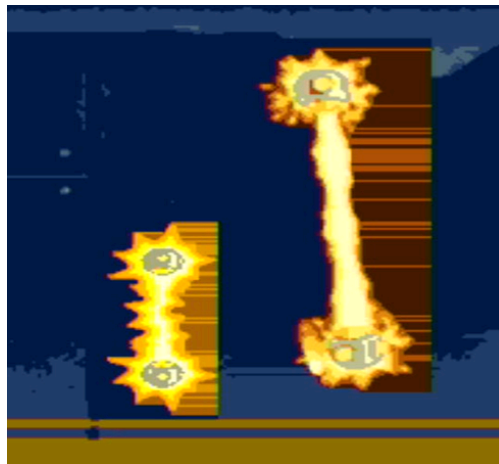


Figure 7 - Random lining issue for incorrect ROM readings

6. Document the Design Resources and Statistics from the lab manual

LUT	3344
DSP	9
Memory (BRAM)	68.5
Flip-Flop	2864
Frequency	$1/(10-1.707) = 120$ MHz
Static Power	0.079 W
Dynamic Power	0.394 W
Total Power	0.472 W
Latches	0

7. Conclusion

Our design worked quite well, with moving obstacles, a moving sprite, and various other components such as collision detection, which allowed us to end the game. We did run into several bugs in the process of making the game, which included latency issues, along with incorrect downsizing for our sprites. We would keep getting extra colored lines next to our sprites, which occurred due to a 2-cycle delay in reading from our sprite ROMS, whereas we only incorporated a one-cycle delay. Furthermore, our gravity did not work at first, which was an issue of signed and unsigned variables. We needed to keep track of these as we needed to increase or decrease our coordinates, and not let them go into the negatives, as sometimes, our sprite would phase through the top of the screen. Along with this, we ran into other issues regarding collision detection and our movement timer on reset, which was caused by the incorrect clock cycle delay and the implementation of the wrong clock speed, causing a mismatch in signals. Overall, our game ended up working quite well, achieving the desired points we had submitted in our original proposal.